UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS INFORMÁTICOS

Un controlador de robot para seguimiento de línea y evadir obstáculos

Robótica y Percepción Computacional

Integrantes:

- Julio Lucero
- Maria Agostina Squillari

Profesor:

- Luis Baumela
- Nik Swoboda
- Roberto Valle

Institución:

- Universidad Politécnica de Madrid

Carrera:

- Grado de Ingeniería Informática

- Fecha: 23/03/2026

Índice

1. Introducción 1

2. Lógica de Seguimiento de Línea (Line Following) 2
   2.1. Problema del Controlador Inicial 2
   2.2. Implementación del Controlador Proporcional Suavizado 2

3. Lógica de Evasión de Obstáculos 3

4. Integración de Comportamientos y Recuperación 3

5. Código Completo 6

6. Conclusiones 8

1. Introducción

El objetivo de este trabajo práctico consiste en modificar y mejorar un controlador base provisto en Pyrobot mas específicamente el Brain control. El controlador original utilizaba una lógica de encendido-apagado para seguir una línea, lo cual resultaba en un movimiento oscilatorio y poco eficiente. Además, carecía de la capacidad para interactuar con su entorno. En este proyecto se han implementado dos mejoras fundamentales: un controlador proporcional para suavizar el seguimiento de la línea y un sistema reactivo para la evasión de obstáculos estáticos utilizando los sensores de rango sonares del robot.

2. Lógica de Seguimiento de Línea (Line Following)

2.1. Problema del Controlador Inicial

El código base proporcionado evaluaba la desviación del robot respecto a la línea (**variable error**) y aplicaba velocidades de giro máximas con las variables **HARD_RIGHT** o **HARD_LEFT** dependiendo de si el error era positivo o negativo. Esto generaba un comportamiento de **zig-zag** muy marcado, tal como se describía en la presentación de algoritmos de control de la asignatura.

2.2. Implementación del Controlador Proporcional Suavizado

Para resolver este problema y obtener una trayectoria fluida, se sustituyó el bloque On-Off por un Controlador Proporcional (**P**), basando la lógica en los conceptos teóricos de los sistemas de control realimentados presentados en clase.

En lugar de girar de manera binaria, la velocidad de giro (**tv**) ahora es proporcional al margen de error detectado por la cámara. Para evitar oscilaciones innecesarias en las rectas, se aplicó un factor de atenuación de 0.8:

```
tv = -0.8 * error
```

Listing 1: Giro Proporcional suavizado basado en el error

De este modo, si el error es pequeño, la corrección es casi imperceptible. Si la desviación es grande, el giro será pronunciado pero controlado.

Adicionalmente, se implementó un ajuste dinámico de la velocidad de avance (**fv**). Si el robot detecta que debe hacer un giro brusco, reduce gradualmente su velocidad lineal para evitar salirse de la pista por inercia, manteniendo un avance mínimo de seguridad:

```
fv = max(0.15, 0.8 - abs(error))
self.move(fv, tv)
```

Listing 2: Dinámica de frenado gradual en curvas

Figura 1 y 2 muestran el seguimiento fluido en línea recta:

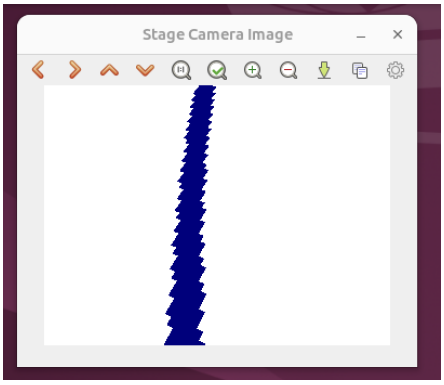

Figura 1: Stage Camera Image (straight line)

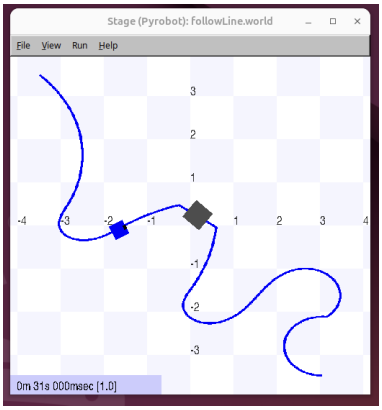

Figura 2: Trayectoria Fluida del Robot en Línea Recta

Figura 3 y 4 demuestran el seguimiento estable en una línea curva:

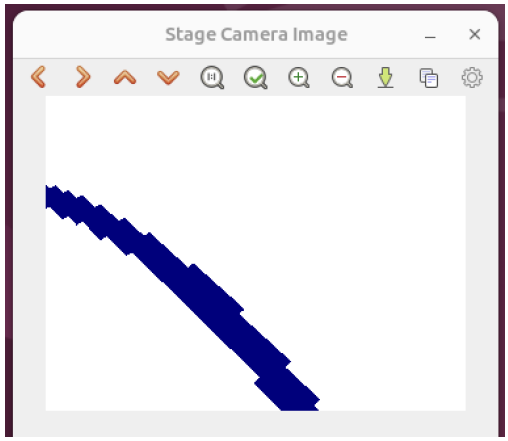

Figura 3: Stage Camera Image (curved line)

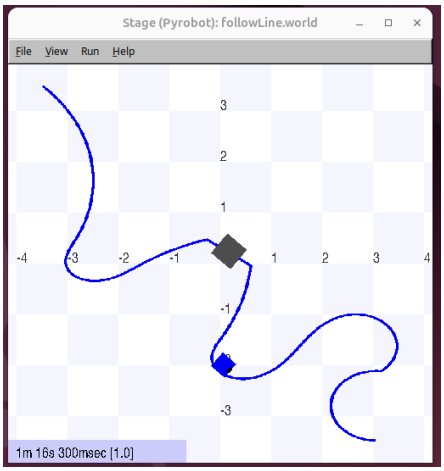

Figura 4: Trayectoria Fluida del Robot en Curva

3. Lógica de Evasión de Obstáculos

Para cumplir con el segundo requerimiento de la práctica, se dotó al robot de la capacidad de evitar cajas estáticas en su camino. Para ello, se utilizó el sensor de rango, extrayendo las lecturas de los 4 sensores estrictamente frontales (índices 2, 3, 4 y 5) y calculando la distancia mínima:

```
front_distances = [self.robot.range[i].distance() for i in range(2, 6)]
min_front_dist = min(front_distances)
```

Listing 3: Extracción de lecturas de sensores frontales (2 al 5)

En versiones iniciales del código, el robot giraba de forma demasiado brusca alejándose excesivamente de la pista. Para solucionar esto, se estableció el umbral de seguridad a **0.35 metros** y se rediseñó la maniobra. Ahora, cuando el obstáculo cruza este umbral, el robot no se detiene a cero ni realiza un giro estático, sino que avanza ligeramente mientras ejecuta un giro pronunciado, trazando una curva de salida suave.

4. Integración de Comportamientos y Recuperación

Para lograr combinar ambas lógicas de forma fluida y evitar que el robot se abra demasiado al esquivar, se estructuró la función $\mathrm{step(self)}$ implementando una máquina de estados sencilla basada en pasos ($\mathrm{avoidance_steps}$) y una jerarquía de prioridades mediante $\mathrm{if\text{-}elif\text{-}else}$:

1. **Prioridad 1 - Seguridad e inicio de evasión:**
   Si hay un obstáculo a menos de 0.35 metros, el robot inicia la maniobra. Avanza lentamente (0.15) y gira a la izquierda (1.0), trazando una curva suave hacia afuera. Se inicializa el contador de pasos en 15.

2. **Prioridad 2 - Maniobra envolvente (Abrazar el obstáculo):**
   Mientras el contador avoidance_steps sea mayor a cero, el robot ejecuta una curva muy abierta hacia la derecha. Esto permite rodear la caja fluidamente sin alejarse demasiado de la trayectoria de la pista.

3. **Prioridad 3 - Búsqueda y Recuperación:**
   Al esquivar, la cámara pierde la línea inferior. Una vez superada la maniobra envolvente, el robot utiliza la memoria del último error registrado (last_error) para trazar un arco de avance que lo reincorpore a la línea en la dirección correcta.

4. **Prioridad 4 - Navegación normal:**
   Si no hay obstáculos cerca y la cámara detecta la línea, se ejecuta el Controlador Proporcional suavizado descrito en la sección 2.

```python
if min_front_dist < 0.35:
    # Evasión inicial: Curva suave hacia afuera
    self.move(0.15, 1.0)
    self.avoidance_steps = 15
    self.last_error = 1.0

elif self.avoidance_steps > 0:
    # Maniobra envolvente para "abrazar" la caja
    self.move(self.MED_FORWARD, -0.25)
    self.avoidance_steps -= 1

elif not foundLine:
    # Recuperación de la línea perdida mediante arcos
    if self.last_error > 0:
        self.move(0.2, -0.8) # Arco suave a la derecha
    else:
        self.move(0.2, 0.8)  # Arco suave a la izquierda

else:
    # Seguimiento de línea con Controlador P suavizado
    self.last_error = error
    tv = -0.8 * error
    fv = max(0.15, 0.8 - abs(error))
    self.move(fv, tv)
```

Listing 4: Lógica combinada de Evasión y Seguimiento Fluido

Figura 5 y 6 demuestran la evasion de obstaculo y como retoma la linea despues de hacer la evasion:

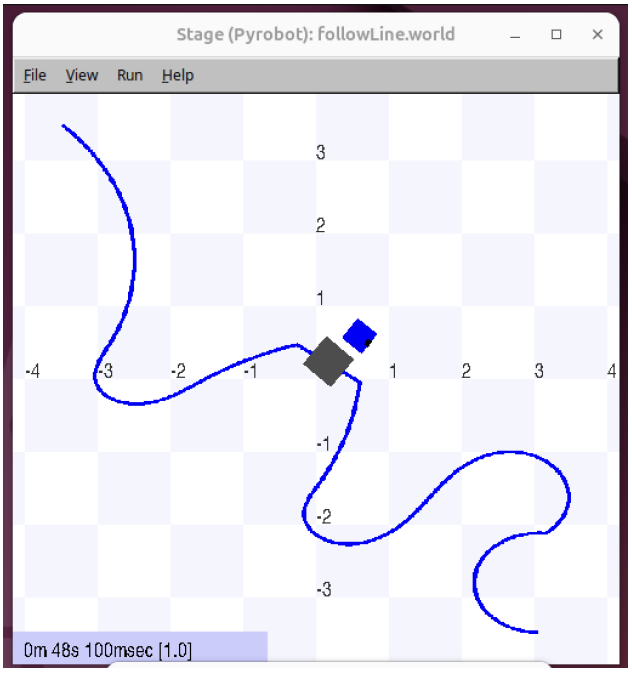

Figura 5: Evasión de Obstáculo

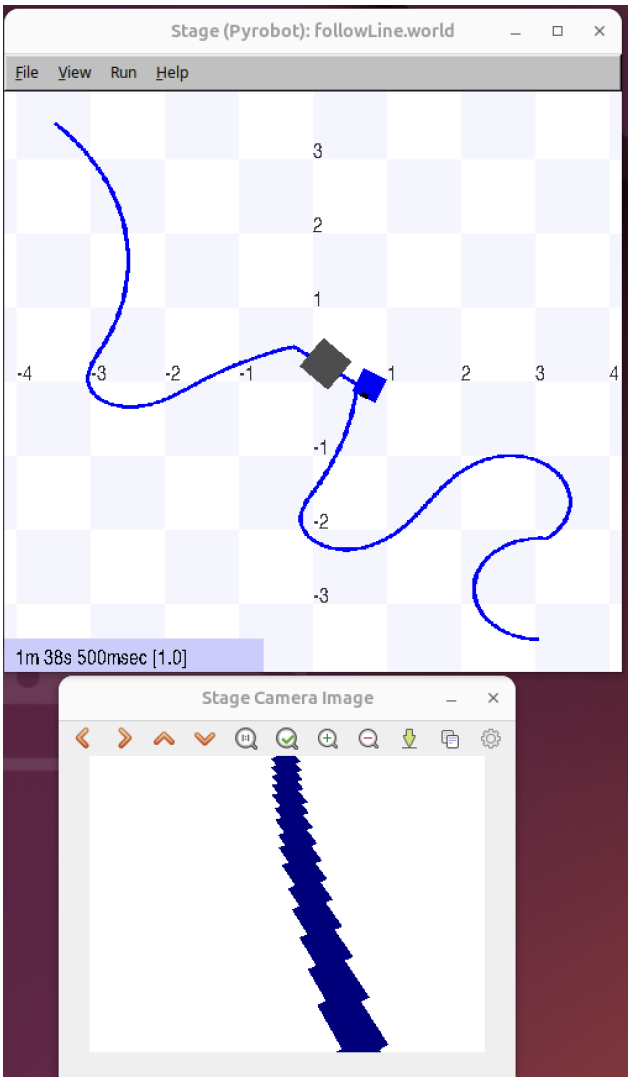

Figura 6: Retoma de la Linea

5. Código Completo

```python
from pyrobot.brain import Brain
import cv2
from pyrobot.tools.followLineTools import findLineDeviation

class BrainFollowLine(Brain):

    SLOW_FORWARD = 0.1
    MED_FORWARD = 0.5

    def setup(self):
        self.last_error = 0
        self.avoidance_steps = 0

    def destroy(self):
        cv2.destroyAllWindows()

    def step(self):
        cv_image = self.robot.getImage()
        cv2.imshow("Stage Camera Image", cv_image)
        cv2.waitKey(1)

        imageGray = cv2.cvtColor(cv_image, cv2.COLOR_BGR2GRAY)
        foundLine, error = findLineDeviation(imageGray)

        front_distances = [self.robot.range[i].distance() for i in range(2, 6)]
        min_front_dist = min(front_distances)

        # 1. EVASIÓN DE OBSTÁCULOS (Transición Fluida)
        # Aumentamos un pelín la distancia de detección para tener espacio de hacer la curva
        if min_front_dist < 0.35:
            # FLUCTUACIÓN 1: En lugar de frenar a cero, avanzamos a 0.15 mientras giramos fuerte.
            # Esto dibuja una curva suave hacia afuera en lugar de un quiebre en "V".
            self.move(0.15, 1.0)
            self.avoidance_steps = 15
            self.last_error = 1.0

        elif self.avoidance_steps > 0:
            # FLUCTUACIÓN 2: En lugar de ir recto, hacemos una curva muy abierta a la derecha.
            # Esto hace que el robot empiece a "abrazar" o envolver el obstáculo con fluidez.
            self.move(self.MED_FORWARD, -0.25)
            self.avoidance_steps -= 1

        elif not foundLine:
            # 3. RECUPERACIÓN DE LÍNEA
            # Mantenemos un poco de avance (0.2) para que la reincorporación
```

```
sea en arco.
            if self.last_error > 0:
                self.move(0.2, -0.8) # Arco suave a la derecha
            else:
                self.move(0.2, 0.8)  # Arco suave a la izquierda

        else:
            # 4. SEGUIMIENTO DE LÍNEA FLUIDO
            self.last_error = error

            # FLUCTUACIÓN 3: Suavizamos el controlador proporcional.
            # Al multiplicar por 0.8 en vez de 1.0, el robot zigzaguea menos
en las rectas.
            tv = -0.8 * error

            # El frenado en curvas ahora es más gradual.
            fv = max(0.15, 0.8 - abs(error))

            self.move(fv, tv)

def INIT(engine):
    assert (engine.robot.requires("range-sensor") and
            engine.robot.requires("continuous-movement"))

    return BrainFollowLine('BrainFollowLine', engine)
```

Listing 5: Codigo Completo

6. Conclusiones

El desarrollo de este trabajo práctico permitió cumplir satisfactoriamente con los objetivos planteados, logrando una mejora sustancial y medible en el desempeño del controlador base de Pyrobot. La transición de una lógica de control discreta **On-Off** a un Controlador Proporcional **P** demostró la eficacia de los sistemas de control realimentados para mitigar el comportamiento oscilatorio. Gracias a la atenuación del error y al ajuste dinámico de la velocidad lineal, se obtuvo una trayectoria de seguimiento de línea notablemente más fluida, estable y capaz de adaptarse a curvas pronunciadas sin perder el control por inercia.

Adicionalmente, la incorporación de un sistema reactivo mediante el uso de sensores de rango ultrasónicos dotó al robot de autonomía frente a entornos dinámicos o parcialmente desconocidos. El diseño de la maniobra de evasión resultó particularmente exitoso, al evitar detenciones a cero y optar por trayectorias curvas y envolventes, se mantuvo la fluidez cinemática del sistema.

Finalmente, la integración de estos comportamientos a través de una máquina de estados basada en jerarquías garantizó una convivencia armónica entre la navegación, la evasión y la recuperación de la línea.